DESAFIO VERX · DOCUMENTAÇÃO DE ARQUITETURA

Decisões Técnicas e Requisitos Não Funcionais

Como ler este documento

Cada decisão abaixo responde a cinco perguntas:

1. qual problema está sendo resolvido;
2. por que essa escolha foi feita;
3. qual trade-off ela assume;
4. qual limitação ainda existe;
5. quando faria sentido evoluir.

Decisões técnicas

Separação entre `ledger-service` e `daily-balance-service`

Problema que resolve Separar a responsabilidade da escrita transacional da responsabilidade de consulta agregada.

Por que foi escolhida Permite tratar escrita e leitura como contextos diferentes, com prioridades diferentes e ciclos de evolução distintos.

Trade-off Introduz consistência eventual e mais componentes para operar.

Limitação atual Apesar da separação lógica, ainda existe compartilhamento do mesmo banco físico.

Quando evoluir Quando houver necessidade de escalar ou isolar operação por contexto de forma independente.

Clean Architecture

Problema que resolve Evitar acoplamento excessivo entre regra de negócio e detalhes de framework, banco ou fila.

Por que foi escolhida Deixa a intenção arquitetural mais legível no case e facilita testar casos de uso sem depender de infraestrutura real.

Trade-off Exige mais classes, portas e adaptadores, o que aumenta o volume estrutural do código.

Limitação atual Para um caso pequeno, parte dessa estrutura é mais arquitetural do que operacional.

Quando evoluir Sempre que o domínio crescer, a separação tende a se pagar ainda mais.

PostgreSQL

Problema que resolve Persistir dados transacionais com integridade, consultas simples e suporte a transações locais.

Por que foi escolhido É um banco relacional maduro, com boa ergonomia para ACID, índices, constraints e operação local simples.

Trade-off Compartilhar o mesmo banco entre contextos simplifica o case, mas reduz isolamento operacional.

Limitação atual Não há separação física entre `ledger` e `balance`.

Quando evoluir Quando a carga, o risco operacional ou a autonomia entre domínios justificar.

SQS / ElasticMQ

Problema que resolve Desacoplar o produtor da consolidação de saldo e absorver processamento assíncrono.

Por que foi escolhido SQS é um mecanismo simples e conhecido para integrações assíncronas; ElasticMQ permite simular o comportamento localmente.

Trade-off A entrega é at-least-once, o que exige idempotência do consumidor.

Limitação atual Ainda não existe uma camada completa de retry operacional e DLQ.

Quando evoluir Quando houver necessidade de monitorar falhas persistentes, poison messages e reprocessamento controlado.

Transactional Outbox

Problema que resolve Evitar perda de evento quando a escrita no banco ocorre, mas a publicação externa falha.

Por que foi escolhido Permite preservar atomicidade entre o fato de negócio persistido e o registro do evento a ser publicado.

Trade-off Adiciona um fluxo extra de publicação e uma nova tabela para gerenciar estado.

Limitação atual O publisher atual não faz claim/lock de eventos em concorrência.

Quando evoluir Ao rodar múltiplas instâncias do publisher ou exigir maior robustez operacional.

Idempotência na entrada por `requestId`

Problema que resolve Evitar duplicidade causada por retry do cliente, timeout ou reenvio da mesma requisição.

Por que foi escolhida É uma proteção simples e eficaz para a borda de escrita.

Trade-off Exige persistência adicional e definição de responsabilidade do cliente em fornecer um identificador único.

Limitação atual Não há expiração ou políticas adicionais para o armazenamento desse identificador.

Quando evoluir Se houver necessidade de governança de retenção ou proteções antiabuso mais sofisticadas.

Idempotência no consumidor por `processed_messages`

Problema que resolve Evitar aplicar o mesmo evento mais de uma vez na projeção de saldo.

Por que foi escolhida A fila pode entregar a mesma mensagem novamente; essa tabela protege a leitura contra duplicidade.

Trade-off Adiciona persistência e uma verificação extra por mensagem.

Limitação atual A estratégia concorrente ainda depende de um desenho simples de leitura seguida de gravação.

Quando evoluir Quando houver processamento realmente paralelo em alta escala e for necessário endurecer o controle concorrente.

Read model materializado `daily_balances`

Problema que resolve Responder consulta de saldo diário sem recalcular o histórico inteiro a cada chamada.

Por que foi escolhido Reduz custo de leitura, simplifica a API de consulta e deixa explícito o desacoplamento entre escrita e leitura.

Trade-off O dado consultado pode atrasar em relação ao ledger.

Limitação atual Não há SLA formal de latência entre escrita e consolidação.

Quando evoluir Quando houver demanda por outras projeções, mais granularidade temporal ou consultas analíticas.

Consistência eventual

Problema que resolve Permitir que a escrita não dependa da consolidação síncrona do saldo.

Por que foi escolhida Melhora desacoplamento, reduz latência da escrita e reforça a independência entre contextos.

Trade-off O saldo diário pode não refletir imediatamente um lançamento recém-criado.

Limitação atual Não há mecanismo formal de reconciliação ou SLO de atraso documentado.

Quando evoluir Quando a operação exigir metas claras de latência de processamento e observabilidade de backlog.

Requisitos não funcionais

Matriz de maturidade

NFR	Estado atual	Evidência	Observação
Escalabilidade	Parcial	separação de leitura/escrita e fila assíncrona	banco ainda compartilhado e polling simples
Resiliência	Parcial	outbox, idempotência e processamento assíncrono	sem DLQ, retry formal ou lock robusto
Performance	Parcial	read model materializado e consulta simples	sem baseline de latência e sem teste de carga executado aqui
Consistência	Atendida dentro do escopo	transação local no ledger e consistência eventual entre contextos	intencionalmente não é imediata
Observabilidade	Básica	health, logger, correlationId, exception filter	sem métricas, tracing e dashboards
Segurança	Básica	`x-api-key` opcional e validação de entrada	sem autenticação forte, rate limit ou segredo gerenciado

Escalabilidade

O que a solução já faz

- separa escrita e leitura em serviços distintos;
- usa fila para desacoplar produtor e consumidor;
- consulta o saldo por uma projeção pré-computada;
- reduz custo de leitura em relação ao recálculo do histórico.

O que ainda falta

- escala independente com isolamento físico por contexto;
- estratégia explícita de particionamento ou throughput;
- políticas mais robustas para concorrência no publisher e no consumer.

Resiliência

O que a solução já faz

- protege a escrita com transação local;
- usa outbox para reduzir risco entre banco e fila;
- trata duplicidade tanto na entrada quanto no consumidor;
- não apaga a mensagem antes do processamento bem-sucedido.

O que ainda falta

- retry controlado;
- DLQ;
- indicadores operacionais de falha;
- estratégia de recuperação ou reconciliação.

Performance

O que a solução já faz

- leitura em tabela agregada;
- índices para consulta principal;
- escrita e publicação desacopladas;
- processamento simples do saldo diário.

O que ainda falta

- baseline de latência;
- medição de throughput;
- teste automatizado com infraestrutura real e carga;
- tuning de polling e paralelismo.

Consistência

O que a solução já faz

- consistência forte na transação local do ledger;
- consistência eventual entre ledger e saldo diário;
- tolerância a entrega repetida por idempotência.

Limitação assumida

O cliente pode observar um lançamento no ledger antes de observar o reflexo no saldo diário.

Isso é esperado na arquitetura atual e deve estar claro na documentação funcional.

Observabilidade

O que a solução já faz

- `GET /health` em ambos os serviços;
- logger simples;
- `correlationId` por middleware;
- filtro global de exceção com resposta padronizada.

O que ainda falta

- métricas;

- tracing distribuído;
- alarmes;
- dashboards;
- SLOs e visão operacional de backlog.

Segurança

O que a solução já faz

- validação de entrada com `ValidationPipe`;
- proteção opcional por `x-api-key`;
- encapsulamento de erros conhecidos por `AppException`.

O que ainda falta

- autenticação forte;
- rate limiting;
- gestão de segredos;
- auditoria e políticas de autorização mais refinadas.